

Home Lab Project Week 6, Day 3

Task:

Analyze the captured scan traffic in Wireshark. Identify the TCP SYN, ACK, and RST flags and determine how the port-scanning activity appears from a network defender's perspective.

Analyzing Scan Traffic — TCP Flags & the Defender's View

Engagement: Home Lab Internal Network Assessment **Lab log:** Week 6, Day 3 **Target:** Ubuntu 192.168.78.5 · **Tester:** Kali Linux 192.168.78.3 **Prepared by:** [Your name], DAE Cybersecurity Program **Date:** [Fill in]

1. Objective

My objective was to analyze the captured Nmap scan traffic in Wireshark, identify the TCP SYN, ACK, and RST flags, and determine how the port-scanning activity appears from a network defender's perspective.

2. TCP Flags — Background

TCP uses control flags in each packet's header to manage connections. Three matter most for this analysis:

Flag	Name	Purpose
SYN	Synchronize	Starts a connection — the first packet of the three-way handshake
ACK	Acknowledge	Confirms receipt of data; present in the handshake and all established traffic
RST	Reset	Abruptly terminates or refuses a connection

A normal, complete TCP connection uses the **three-way handshake**:

Client → Server : SYN

Server → Client : SYN, ACK

Client → Server : ACK (connection established)

A SYN scan deliberately never reaches that third step.

3. Reading the Flags in Wireshark

I read the flags two ways: the **Info** column shows them in brackets (e.g. [SYN], [SYN, ACK], [RST, ACK]), and expanding **Transmission Control Protocol** → **Flags** in the detail pane shows each bit individually. Each flag combination also has a hex value I can filter on.

What it is	Flags	Info column	Hex
Scan probe (Kali → target)	SYN	[SYN]	0x00 2
Open port reply (target → Kali)	SYN + ACK	[SYN, ACK]	0x01 2
Closed port reply (target → Kali)	RST + ACK	[RST, ACK]	0x01 4
Kali aborting an open port	RST	[RST]	0x00 4

Display filters I used

tcp.flags.syn == 1 && tcp.flags.ack == 0 # SYN probes from Kali

tcp.flags.syn == 1 && tcp.flags.ack == 1 # SYN-ACK from open ports (21, 23)

tcp.flags.reset == 1 # all RST packets

4. What Each Flag Shows in This Scan

- **SYN** — Kali sent a flood of SYN packets, one to each of ~1,000 ports, in a very short time. These are the scan probes.

- **SYN, ACK** — only the two open ports (**21/FTP** and **23/telnet**) replied with SYN-ACK, indicating they are listening and willing to connect.
- **RST, ACK** — the 998 closed ports each replied with RST-ACK, telling the sender nothing is listening.
- **RST** — for the two open ports, Kali replied with a bare RST instead of the final ACK, tearing the connection down before it completed. This is the half-open signature of a SYN scan.

So the entire scan is visible purely through the flag patterns: many outbound SYNs, a wall of RST-ACK replies, two SYN-ACK replies, and two RST teardowns.

5. The Network Defender's Perspective

This is how the same traffic looks to someone defending the target.

5.1 What makes it obviously abnormal

Normal traffic involves a host talking to a *few* services and completing full handshakes that carry real data. This capture shows the opposite:

- **One source, many destinations.** A single IP (**192.168.78.3**) contacted ~1,000 different ports on one host.
- **In a very short time.** All of it happened in seconds — far faster than any legitimate user behavior.
- **Connections that never complete.** Hundreds of half-open attempts with no data transfer. A high ratio of SYN packets to completed sessions is a classic scan indicator.
- **A burst of RST from the target.** The target generating ~1,000 RSTs in a moment is itself an anomaly worth alerting on.

5.2 How a defender would detect it

- **IDS/IPS (e.g. Snort, Suricata).** These tools have port-scan detection that counts connection attempts from one source across many ports within a time window and raises an alert when a threshold is crossed.
- **Firewall / connection logs.** A wall of log entries from one IP hitting sequential or numerous ports stands out immediately.
- **SIEM correlation.** Aggregated logs would flag the "one source → many ports → short time" pattern as reconnaissance.
- **Rate/threshold monitoring.** Any rule like "more than N new connections from one host in M seconds" would trip.

5.3 What it signals and how a defender responds

A port scan is almost always **reconnaissance** — an attacker mapping which services are exposed before choosing an attack. Typical defensive responses:

- **Alert** the security team and log the source IP for investigation.
- **Block or rate-limit** the offending IP (e.g. with a firewall rule or a tool like fail2ban).
- **Reduce the attack surface** by closing unnecessary ports so future scans reveal less.
- **Segment the network** so a scanner on one segment cannot freely probe others.

5.4 A note on stealth

A SYN scan is quieter than a full-connect scan because it never completes the handshake, so it may not appear in an application's own connection logs. However, it is still very detectable at the **network** layer by the volume-and-pattern methods above — stealthy against one log source is not the same as invisible.

6. Observations

- All three target flags (SYN, ACK, RST) were clearly identifiable in the capture, and their combinations told the whole story of the scan.
 - The open-port SYN-ACK responses matched my Day 1 findings exactly (ports 21 and 23).
 - From a defender's viewpoint, this traffic is a textbook reconnaissance scan: one source, ~1,000 ports, sub-second timing, and many incomplete connections — precisely the pattern intrusion-detection systems are built to catch.
-

7. Evidence Log

- [] Screenshot: a [SYN] probe with the **Flags** section expanded in the detail pane
 - [] Screenshot: a [SYN, ACK] reply from port 21 or 23
 - [] Screenshot: a [RST, ACK] reply from a closed port
 - [] Screenshot: the [RST] teardown from Kali to an open port
 - [] Screenshot: **Statistics** → **Conversations** showing the one-to-many fan-out
-

This document records authorized security testing I performed in a controlled lab environment.